GraphQL Technical Interview Guide

Comprehensive Questions, Practical Answers & Architectural Standards

2026 Edition

LEVEL 1: ENTRY & JUNIOR DEVELOPER FUNDAMENTALS

CORE CONCEPTS & SYNTAX

Q1: What is GraphQL, and how does it differ fundamentally from REST?

GraphQL is an open-source data query and manipulation language for APIs, alongside a runtime for fulfilling queries with existing data. Unlike REST—which exposes multiple fixed endpoints mapped to static resource structures—GraphQL exposes a single endpoint where clients submit declarative requests specifying exactly the fields they require.

- **Endpoints**
- Multiple (e.g., `/users`, `/posts/1`)
- Single HTTP endpoint (e.g., `/graphql`)
- **Data Fetching**
- Over-fetching / Under-fetching common
- Exact shape requested is returned
- **Schema & Typing**
- Optional (OpenAPI/Swagger docs)
- Mandatory strongly typed Schema (SDL)

Feature	REST Architecture	GraphQL Specification

Q2: Explain the three core operation types in GraphQL.

GraphQL supports three root operation types defined in the schema:

- **Query:** Read-only operations used to fetch data without causing side effects. Multiple query root fields execute concurrently.
- **Mutation:** Operations used to create, update, or delete data (write operations). Root fields in a mutation execute sequentially to ensure transactional safety.
- **Subscription:** Long-lived operational connections (usually over WebSockets) where the server pushes real-time event updates to connected clients whenever data changes.

Q3: What are scalar types in GraphQL, and what are the built-in defaults?

Scalar types represent primitive, single values at the leaves of a GraphQL query tree. They parse directly to concrete primitive data types in the response payload. The five built-in standard scalar types are:

`Int` (Signed 32-bit int), `Float` (Double-precision float), `String` (UTF-8 character sequence), `Boolean` (true/false), `ID` (Unique text identifier)

Q4: What is the role of a Resolver function, and what arguments does it receive?

A **Resolver** is a backend function responsible for returning data for a specific field in the schema. When a query is parsed and validated, the GraphQL engine invokes the corresponding resolver for each field requested. Every resolver receives 4 positional arguments:

```
const resolver = (parent, args, context, info) => { ... }
```

- `parent` (or `root`): The return value of the previous resolver up the parent tree.
- `args`: The parameters passed into the specific field inside the GraphQL query string.
- `context`: Shared operational context across all resolvers (auth user, database client, loaders).
- `info`: AST operational state, field name, schema metadata, and path execution details.

LEVEL 2: MID-LEVEL SOFTWARE ENGINEER

PERFORMANCE, SCHEMA DESIGN & ERRORS

Q5: What is the N+1 problem in GraphQL, and how is it solved using DataLoader?

The **N+1 Problem** occurs because GraphQL resolvers execute independently per field. If a query requests a list of `N` posts along with each post's `author`, a naive resolver fires 1 SQL query to fetch the posts, plus `N` separate SQL queries to fetch each author, causing server bottlenecks.

Solution: Use **DataLoader** (a batching and caching utility). DataLoader coalesces individual fetch requests made during a single tick of the JavaScript event loop into a single batch query (e.g., `SELECT * FROM users WHERE id IN (...)`), reducing `N+1` database calls down to exactly 2.

Q6: Compare Union types vs. Interface types with practical SDL examples.

Both are abstract types used when a field can return multiple concrete types. An **Interface** enforces a shared contract (common fields), whereas a **Union** combines unrelated types without common fields.

Interface (Shared Contract):

```
interface Media {
  id: ID!
  title: String!
}
type Book implements Media {
  id: ID!
  title: String!
  pages: Int
}
```

Union (Unrelated Types):

```
union SearchResult = Book | User

type Query {
  search(q: String!): [SearchResult!]!
}
```

Q7: How does error handling work in GraphQL? Contrast partial success with REST errors.

Unlike REST APIs that rely heavily on HTTP status codes (e.g., 404, 500), GraphQL servers almost always return `HTTP 200 OK`. The execution response payload separates standard data from field execution errors via two top-level keys: `data` and `errors`.

Partial Success: If one resolver fails in a nested selection, the failed field resolves to `null` (if nullable) and populates the top-level `errors` array with path, location, and message details. Other sibling resolvers continue executing normally and return valid data in the same response.

Q8: Explain the difference between Offset-based and Cursor-based Pagination.

Offset Pagination (using `limit` and `offset`) is simple but vulnerable to duplicate/skipped records when items are inserted or deleted between requests. It also degrades in SQL performance at high offsets.

Cursor-based Pagination (Relay spec using `first`, `after`, `edges`, `node`) uses an opaque string cursor pointing to a specific record. It guarantees stable feeds during real-time updates and optimizes indexed database pagination.

LEVEL 3: SENIOR & STAFF TECHNICAL LEAD

DISTRIBUTED ARCHITECTURE & PRODUCTION SECURITY

Q9: What is Apollo Federation, and how does it compare to Schema Stitching?

Both patterns consolidate multiple underlying GraphQL microservices into a unified graph gateway, but they differ significantly in ownership and mechanics:

- **Schema Stitching:** Imperative and centralized. A gateway service imports sub-schemas and defines explicit programmatic glue code to route and transform fields across services.
- **Apollo Federation:** Declarative and distributed. Individual microservices (subgraphs) define their own entity schemas using directives like `@key`, `@external`, and `@provides`. A router automatically composes these subgraphs into a supergraph schema without custom routing code.

Q10: How do you prevent Denial of Service (DoS) attacks on a public GraphQL endpoint?

Because clients can construct arbitrarily nested queries (e.g., `user -> friends -> friends -> friends`), public GraphQL APIs are vulnerable to resource exhaustion. Key defense strategies include:

1. **Max Query Depth Limiting:** Statically analyze the AST before execution and reject any request exceeding a set nesting depth (e.g., max 7 levels).
2. **Query Complexity Analysis:** Assign cost points to schema fields (e.g., heavy list fields = 10 points) and reject queries exceeding a max complexity score per request.
3. **Persisted Queries (Allowlisting):** In production client builds, generate hashes of approved queries. The server only executes pre-approved query hashes, ignoring arbitrary user-submitted strings.

Q11: How do you handle authentication, field-level authorization, and schema evolution?

AuthN & AuthZ: Perform authentication once at the HTTP middleware layer (e.g., parsing JWT tokens) and attach the user context. Handle authorization inside individual field resolvers or via schema directives (e.g., `@auth(requires: ADMIN)`) to guard individual fields dynamically.

Schema Evolution: Never version GraphQL endpoints (e.g., `/v1`, `/v2`). Evolve the schema by marking legacy fields with `@deprecated(reason: "Use newField")`. Track field usage metrics via telemetry tools before removing deprecated fields safety.

Senior Candidate Tip: When answering GraphQL architectural questions, always emphasize *trade-offs*. Highlight operational complexity (gateway composition overhead, distributed tracing), caching challenges (loss of standard HTTP GET caching), and how client-side libraries (Apollo Client, Relay) manage normalized store caches.